

Informe – Taller LSTM: predicción de demanda energética

Oct 7, 2026 · @Sara

1. Objetivo y problema

La mejor arquitectura LSTM predice la demanda eléctrica de la hora siguiente con un error medio de **19.6 MW (MAPE 3.36 %, R² 0.904)**, un 44 % menos de error que repetir la demanda actual.

- **Problema:** con una ventana de n horas consecutivas (12, 24 o 48), predecir la demanda de la hora $t+1$.
- **Dataset:** `dataset_demanda_energia_LSTM_2025_2026.csv`, serie horaria del 2025-01-01 al 2026-12-31 (17 550 filas, 15 columnas). Contiene valores faltantes, anomalías, duplicados, desorden temporal e inconsistencias de texto.
- **Variables:** `timestamp`, `demanda_mw`, `temperatura_c`, `humedad_pct`, `viento_kmh`, `radiacion_wm2`, `precipitacion_mm`, `precio_kwh`, `hora`, `dia_semana`, `fin_semana`, `festivo`, `mes`, `periodo_dia` y `demanda_objetivo` (target).
- **Entregables:** `01_data_cleaning.ipynb` (limpieza), `02_lstm_modeling.ipynb` (modelado y análisis), `app.py` (app web) y este informe.

Condición del taller	Cómo se cumple
No usar información futura como entrada	<code>demanda_objetivo</code> se excluye de las entradas; cada ventana termina en t y el target es $t+1$ (verificado con asserts)
Ordenar cronológicamente antes de crear secuencias	Serie ordenada y verificada: 17 520 horas consecutivas, sin huecos
Split cronológico, sin <code>train_test_split</code> aleatorio	Cortes por fecha: train ene 2025–jun 2026, validación jul–sep 2026, prueba oct–dic 2026
Justificar la limpieza y no inventar valores	Cada regla está justificada; ningún valor se fabrica (sección 2)
Normalizar solo con entrenamiento	<code>StandardScaler.fit</code> solo con filas de entrenamiento

Condición del taller	Cómo se cumple
Mínimo tres arquitecturas LSTM	A. LSTM base, B. LSTM profunda, C. LSTM con mayor capacidad y Dropout
Ventanas de 12, 24 y 48 horas	3 arquitecturas × 3 ventanas = 9 modelos
MAE, MSE, RMSE, MAPE y R ²	Tabla de resultados (sección 6)
Sobreajuste y generalización	Sección 8, con experimentos de Dropout, Early Stopping y neuronas

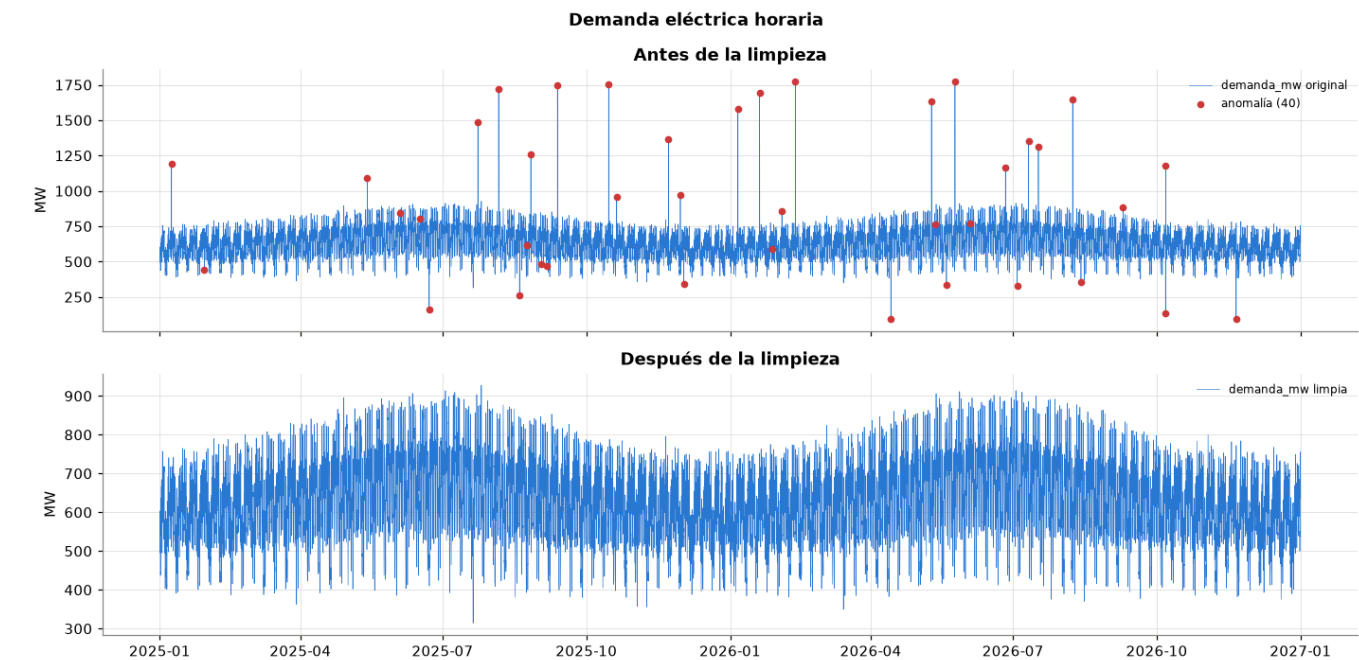
2. Limpieza del dataset

El dataset limpio tiene 17 520 horas consecutivas sin duplicados, sin anomalías y sin faltantes en las entradas; ningún valor se inventó. Los 586 valores modificados quedan registrados uno por uno en `outputs/reports/cleaning_changes.csv`.

Paso	Problema	Regla aplicada	Registros	Justificación
1	Texto inconsistente en <code>periodo_dia</code> ('noche', TARDE, MAÑANA, Mañana)	Quitar espacios, pasar a minúsculas y normalizar Unicode	80	Son la misma categoría escrita distinto; sin unificar crearían categorías falsas
3	Filas duplicadas exactas	Eliminar copias (antes se verificó que no hubiera conflictos)	30	Duplican el peso de una hora y rompen la continuidad de las ventanas
4	Desorden temporal	Ordenar por <code>timestamp</code>	17 520 filas	La LSTM asume que $t+1$ sigue a t
5	<code>periodo_dia</code> que contradice la <code>hora</code>	Derivarlo de la hora: 0–5 madrugada, 6–11 mañana, 12–17 tarde, 18–23 noche	60	Es una etiqueta derivada, no una medición; se deduce sin inventar

Paso	Problema	Regla aplicada	Registros	Justificación
6	Valores físicamente imposibles	Humedad fuera de 0–100, viento < 0 o precio ≤ 0 → marcar como faltante	20	Son imposibles; se eliminan, no se sustituyen
7	Picos de clima y precio (p. ej. –23 °C entre horas de 16 °C)	Z-score modificado > 3.5 sobre la mediana móvil de 25 h, o pico aislado > 3.5 → marcar como faltante	100	Mediana y MAD son robustas a los propios atípicos; 3.5 es el umbral de Iglewicz y Hoaglin
8	Anomalías en <code>demanda_mw</code> (95 MW o 1776 MW)	Reemplazar por <code>demanda_objetivo(t-1)</code>	40	Es la misma medición registrada dos veces en el dataset: es un valor observado, no inventado
9	Faltantes en variables exógenas (vacíos + anomalías)	Repetir el último valor observado (<i>forward fill</i>), máximo 3 h	400	Solo usa el pasado; descartar las ventanas con faltantes perdería el 24 % / 43 % / 68 % de las ventanas de 12 / 24 / 48 h
10	Target vacío en la última hora	No se usa como muestra	1	No existe la hora siguiente en el dataset

Hallazgo clave: `demanda_objetivo(t)` coincide con `demanda_mw(t+1)` en 17 479 de 17 519 horas. Por eso `demanda_objetivo` nunca puede ser entrada: sería información futura. Esa redundancia también permitió detectar las 40 anomalías de demanda, de las cuales 11 no se habrían detectado con un z-score.



Arriba, la serie original con las 40 anomalías de demanda en rojo (entre 95 y 1776 MW);
abajo, la serie limpia, que queda entre 314 y 927 MW.

3. Preparación de los datos

Cada muestra es una ventana de n horas $\times$ 15 variables, y su target es la demanda de la hora siguiente.

Variables de entrada, por cada hora de la ventana:

Grupo	Variables	Tratamiento
Demanda histórica	demanda_mw	estandarizada
Clima	temperatura_c, humedad_pct, viento_kmh, radiacion_wm2, precipitacion_mm	estandarizadas
Mercado	precio_kwh	estandarizada
Calendario binario	fin_semana, festivo	0 / 1
Calendario cíclico	hora, dia_semana, mes $\rightarrow$ seno y coseno	en $[-1, 1]$

- **Target:** `demanda_objetivo` (demanda de $t+1$). No es entrada, porque es información futura.

- **Seno y coseno:** con la hora como entero, las 23 h y las 0 h quedarían "lejos" aunque son horas contiguas; en un círculo quedan juntas.
- **periodo_dia no se usa:** es una función de la hora y no aporta información nueva.

División cronológica, asignada por la hora que se predice:

Conjunto	Horas predichas	Muestras (ventana 24 h)	Uso
Entrenamiento	2025-01-01 → 2026-06-30	13 080	ajustar pesos y normalización
Validación	2026-07-01 → 2026-09-30	2 208	Early Stopping y elección del modelo
Prueba	2026-10-01 → 2026-12-31	2 208	evaluación final, una sola vez

- **Normalización:** `StandardScaler` ajustado solo con las filas de entrenamiento. Validación y prueba se transforman con esos mismos parámetros, así no entra información del futuro a través de la media o la desviación.
- **Secuencias:** para cada hora t , $X = \text{filas } t-n+1 \dots t$ y $y = \text{demanda de } t+1$. Se verificó automáticamente que cada ventana tiene horas consecutivas y termina justo antes de la hora predicha.
- **Comparación justa:** el conjunto de prueba contiene exactamente las mismas horas para las ventanas de 12, 24 y 48 h.

4. Diseño de las arquitecturas

Las tres arquitecturas usan solo capas LSTM, Dense y Dropout. Así, las diferencias de desempeño se atribuyen a dos factores controlados: la profundidad (B frente a A) y la capacidad más la regularización (C frente a B).

Arquitectura	Estructura	Parámetros	Qué pone a prueba
A. LSTM base	<code>LSTM(64) → Dense(1)</code>	20 545	Referencia: la arquitectura más simple que aprovecha la memoria de la LSTM
B. LSTM profunda	<code>LSTM(64, return_sequences=True) → LSTM(32) → Dense(1)</code>	32 929	Si más profundidad mejora el resultado
C. LSTM con mayor capacidad y regularización	<code>LSTM(128, return_sequences=True) → Dropout(0.2) → LSTM(64) → Dropout(0.2) → Dense(32, ReLU) → Dense(1)</code>	125 249	Si más neuronas, controladas con Dropout, mejoran a los modelos pequeños

- **A:** una sola capa recurrente de 64 unidades resume la ventana en su último estado oculto. Una neurona lineal da la predicción (regresión). Entrena rápido y tiene bajo riesgo de sobreajuste.
- **B:** la primera capa devuelve su salida en cada hora (`return_sequences=True`), de modo que la segunda recibe una secuencia completa. La primera capta patrones hora a hora y la segunda los combina en estructuras más largas (la rampa de la mañana, el pico de la tarde).
- **C:** duplica las neuronas de B y añade una capa densa intermedia con ReLU. Como tiene 4 veces más parámetros, es la que más riesgo tiene de memorizar el entrenamiento; por eso lleva Dropout de 0.2 después de cada capa LSTM.

5. Entrenamiento

Los 9 modelos se entrenaron con la misma configuración, y Early Stopping los detuvo entre la época 17 y la 32 de un máximo de 100.

Parámetro	Valor
Optimizador	Adam, learning rate 0.001
Pérdida	MSE sobre el target estandarizado

Parámetro	Valor
Tamaño de batch	64
Épocas máximas	100
Early Stopping	monitorea <code>val_loss</code> , paciencia 10, mejora mínima 0.0001, <code>restore_best_weights=True</code>
Semilla	42 (reproducibilidad)
Hardware	CPU; entrenar todo desde cero toma unos 25 min

- **Early Stopping** detiene el entrenamiento cuando la validación lleva 10 épocas sin mejorar y recupera los pesos de la mejor época. Por eso cada modelo entrena 10 épocas más allá de su mejor punto.
- **Orden de las ventanas:** Keras baraja el orden de las ventanas de entrenamiento en cada época. No contradice la condición de split cronológico: cada ventana conserva su orden interno y todas pertenecen al periodo de entrenamiento.
- **Modelos guardados:** quedan en `models/` . Con `RETRAIN = False` el notebook los carga en 1-2 minutos sin reentrenar.

6. Tabla de resultados

B. LSTM profunda con ventana de 24 h es el mejor modelo: RMSE de 24.68 MW y R^2 de 0.904. Los 9 modelos quedan dentro de 1.5 MW de RMSE entre sí, y todos reducen a casi la mitad el error de la persistencia.

Métricas sobre el conjunto de prueba (oct-dic 2026, 2 208 horas nunca vistas en el entrenamiento). MAE y RMSE en MW, MSE en MW^2 . Épocas = entrenadas (entre paréntesis, la época cuyos pesos restauró Early Stopping).

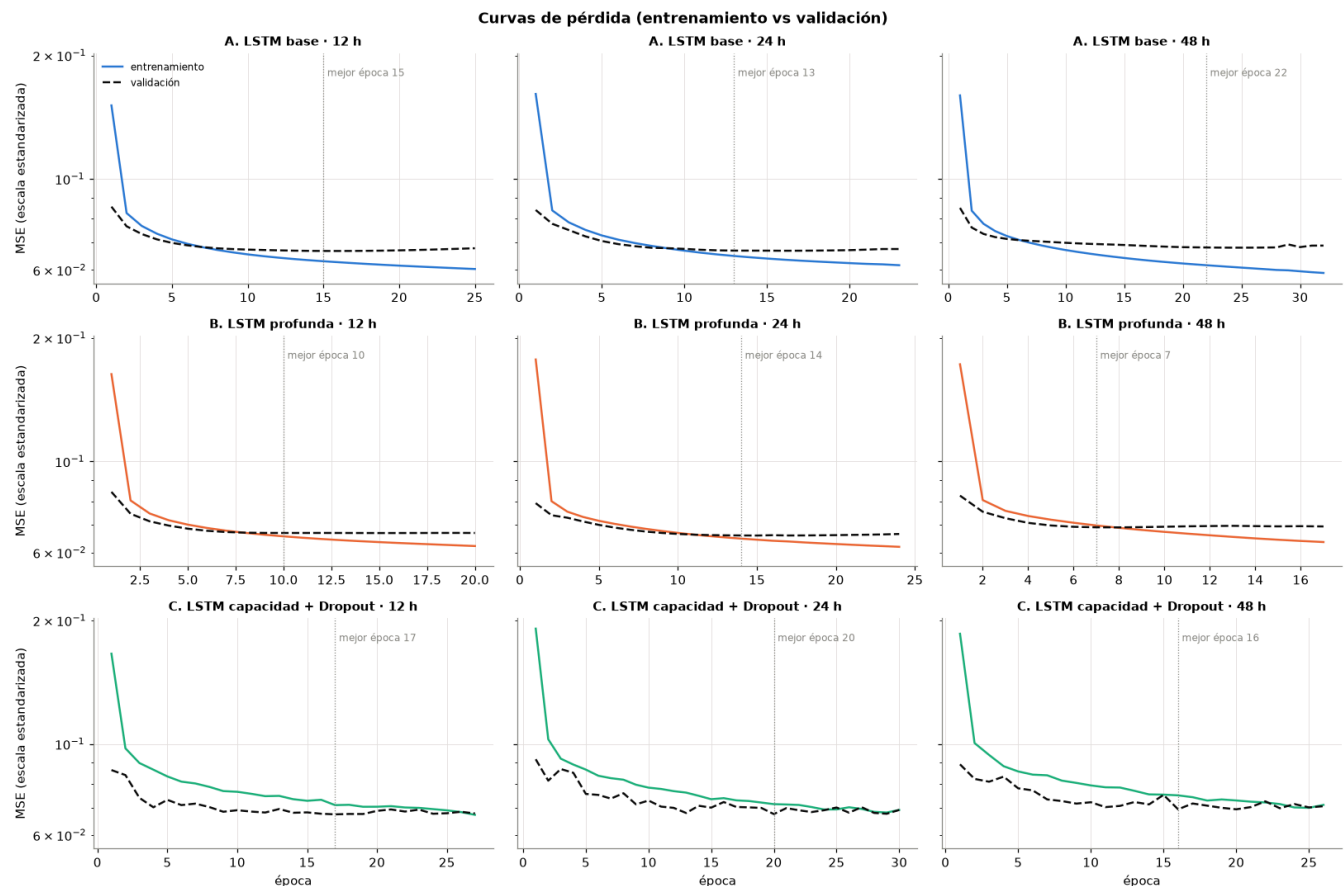
Modelo	Ventana	MAE (MW)	MSE (MW^2)	RMSE (MW)	MAPE (%)	R^2	Épocas
A. LSTM base	12 h	19.84	616.3	24.83	3.39	0.9031	25 (15)
A. LSTM base	24 h	19.75	615.3	24.81	3.38	0.9032	23 (13)
A. LSTM base	48 h	20.21	642.8	25.35	3.47	0.8989	32 (22)
B. LSTM profunda	12 h	20.10	637.2	25.24	3.44	0.8998	20 (10)
B. LSTM profunda	24 h	19.59	609.3	24.68	3.36	0.9042	24 (14)

Modelo	Ventana	MAE (MW)	MSE (MW ²)	RMSE (MW)	MAPE (%)	R ²	Épocas
B. LSTM profunda	48 h	20.57	687.5	26.22	3.54	0.8919	17 (7)
C. LSTM capacidad + Dropout	12 h	19.63	613.4	24.77	3.37	0.9035	27 (17)
C. LSTM capacidad + Dropout	24 h	20.17	635.4	25.21	3.45	0.9001	30 (20)
C. LSTM capacidad + Dropout	48 h	20.50	673.8	25.96	3.54	0.8940	26 (16)
Línea base: persistencia (repetir la hora t)	-	35.30	1956.6	44.23	5.99	0.6920	-
Línea base: misma hora del día anterior	-	51.40	4 716.9	68.68	8.92	0.2580	-

El modelo se eligió por su RMSE en **validación** (25.29 MW, el menor de los 9), no por el de prueba. Así el conjunto de prueba sigue siendo una evaluación independiente.

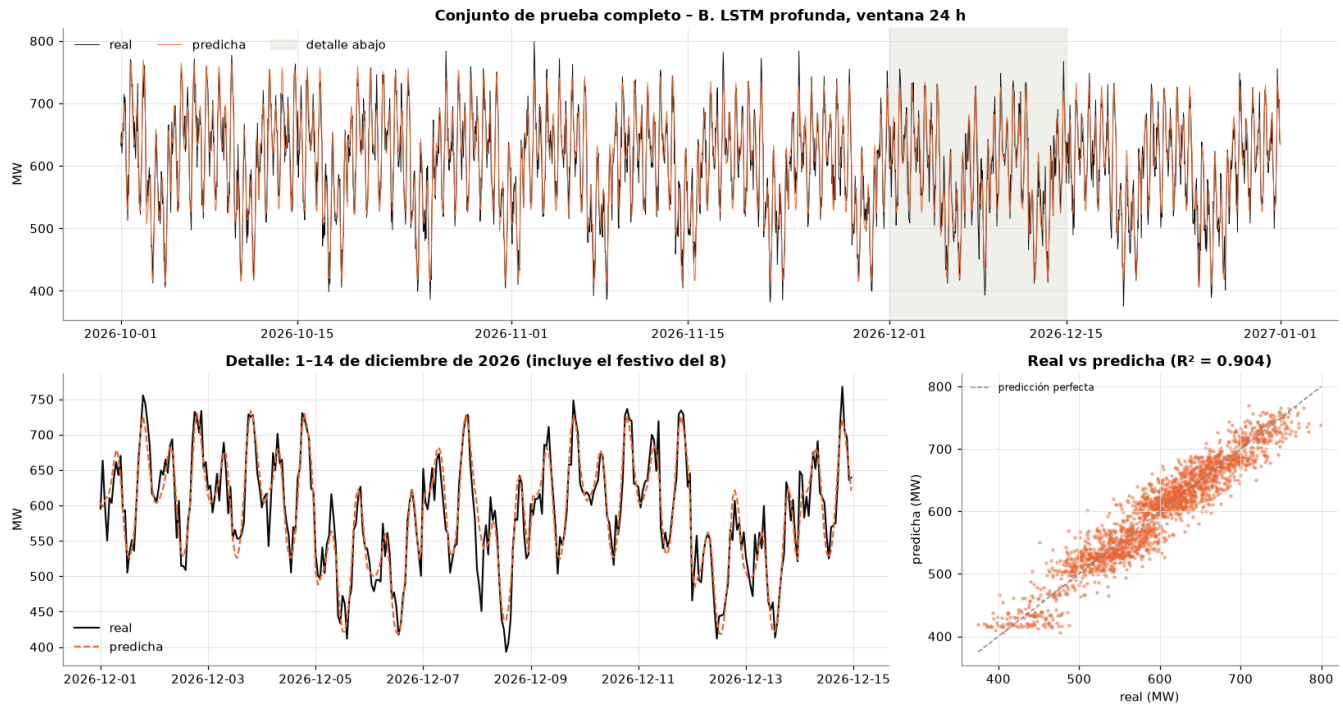
7. Gráficas

7.1 Loss de entrenamiento y validación

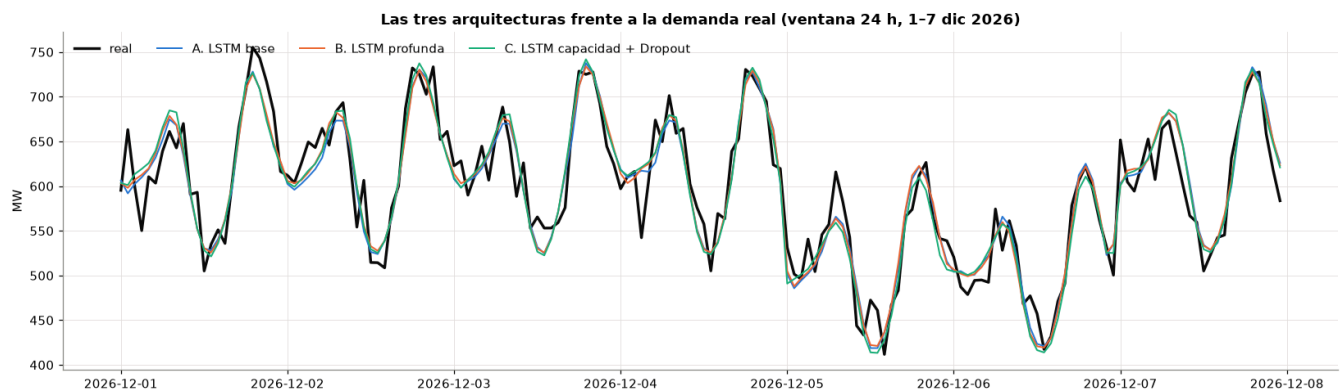


En los 9 modelos la pérdida de validación baja y se estabiliza a las pocas épocas. Early Stopping corta el entrenamiento antes de que empiece a subir; la línea punteada marca la época cuyos pesos se conservan. En C la pérdida de entrenamiento queda por encima de la de validación por efecto del Dropout, que solo actúa al entrenar.

7.2 Demanda real vs. predicha



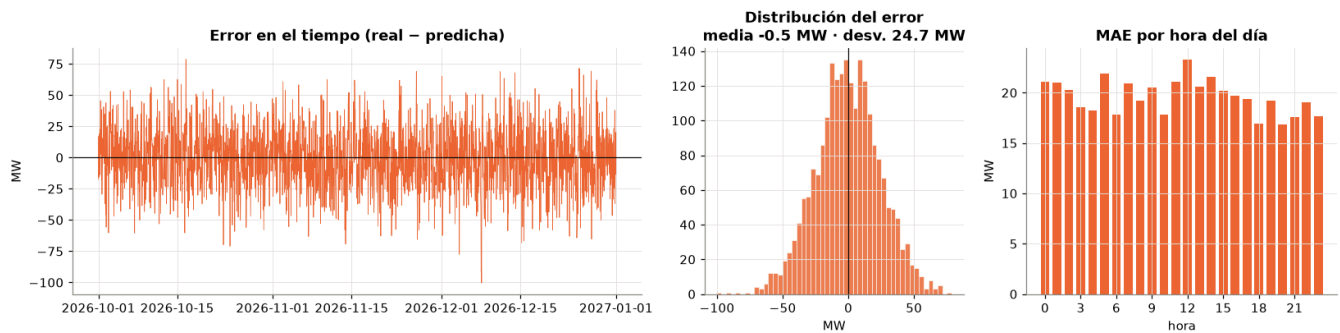
La predicción del mejor modelo (B, 24 h) sigue el ciclo diario y los cambios entre días laborables y fines de semana durante los 3 meses de prueba. El diagrama de dispersión se ajusta a la diagonal ($R^2 = 0.904$). Los mayores desvíos están en picos y valles puntuales, como el festivo del 8 de diciembre.



Con la misma ventana de 24 h, las tres arquitecturas producen curvas casi superpuestas.

7.3 Error de predicción

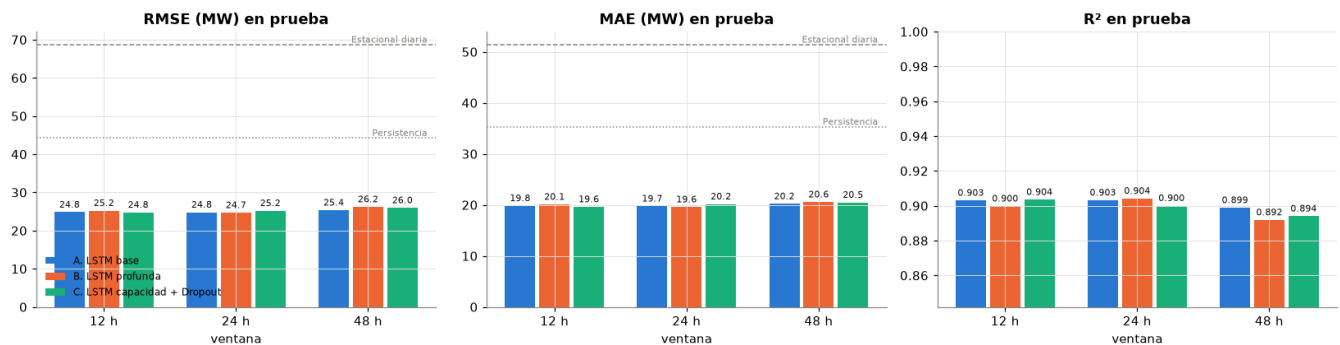
Error de predicción en prueba - B. LSTM profunda, ventana 24 h



El error está centrado en cero (sin sesgo sistemático) y tiene forma aproximadamente normal. Varía poco según la hora del día. En horas festivas el MAE sube a 30.7 MW, frente a 19.3 MW en días normales.

7.4 Comparación de modelos

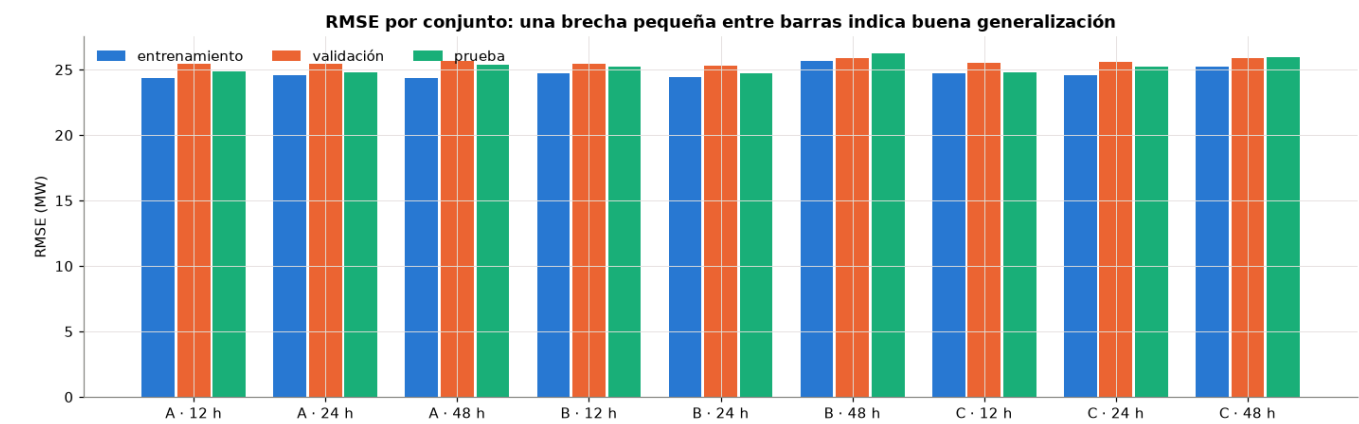
Comparación de modelos (líneas grises = líneas base)



Las 9 combinaciones tienen un error muy parecido y todas están muy por debajo de las líneas base (líneas grises). La ventana de 48 h es la peor en las tres arquitecturas.

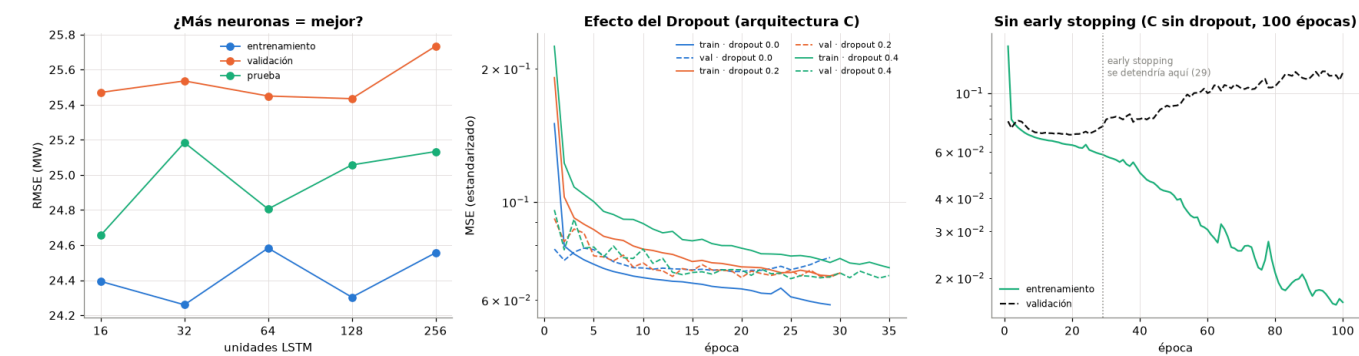
8. Sobreajuste y generalización

Los 9 modelos generalizan bien: su RMSE de validación es solo 1-6 % mayor que el de entrenamiento, y el de prueba es parecido al de validación. Sin Early Stopping, en cambio, el error de prueba sube un 44 %.



Las tres barras de cada modelo (entrenamiento, validación y prueba) tienen casi la misma altura, en torno a 24–26 MW: el modelo rinde igual con datos que nunca vio.

Experimentos complementarios (ventana de 24 h, mismas capas y misma configuración):



Experimento	Configuración	Parámetros	RMSE train (MW)	RMSE validación (MW)	RMSE prueba (MW)	Brecha val/train
Neuronas (A)	16 unidades	2 065	24.39	25.47	24.66	1.044
Neuronas (A)	64 unidades (modelo principal)	20 545	24.58	25.45	24.81	1.035
Neuronas (A)	256 unidades	278 785	24.56	25.73	25.13	1.048
Dropout (C)	0.0	125 249	24.41	25.97	25.68	1.064
Dropout (C)	0.2 (modelo principal)	125 249	24.59	25.56	25.21	1.040
Dropout (C)	0.4	125 249	24.55	25.49	24.89	1.038
Sin Early Stopping	C sin Dropout, 100 épocas	125 249	21.23	34.00	36.90	1.602

- **Sin Early Stopping hay sobreajuste claro.** El error de entrenamiento baja a 21.2 MW, el mejor de todos, pero el de validación sube a 34.0 MW y el de prueba a 36.9 MW. La `val_loss` final es 71 % peor que en la época 19. Early Stopping se habría detenido en la época 29 y conservado la 19.
- **Más Dropout reduce la brecha** entre validación y entrenamiento: de 1.064 sin Dropout a 1.038 con 0.4.
- **Más neuronas no mejoran:** 256 unidades (135 veces más parámetros que 16) dan un error ligeramente mayor.

9. Respuestas a las preguntas de análisis

1. ¿Por qué LSTM es apropiada para este problema?

La demanda es una secuencia con memoria: la próxima hora depende de las anteriores (la rampa de la mañana, el pico de la tarde, la tendencia reciente). Una LSTM procesa la ventana hora por hora, en orden, y mantiene un estado interno que resume ese historial.

Sus compuertas de olvido, entrada y salida deciden qué guardar y qué descartar. Eso evita el gradiente que se desvanece de las RNN simples y le permite aprender dependencias de varias horas. Además, recibe varias variables a la vez (demanda, clima, precio, calendario) y aprende relaciones no lineales entre ellas. En los resultados, la mejor LSTM reduce el RMSE de 44.2 MW (persistencia) a 24.7 MW, un 44 % menos, y sube el R^2 de 0.69 a 0.90.

2. ¿Qué efecto tiene cambiar la ventana temporal?

Con 12 h y 24 h el error es prácticamente el mismo, y 48 h es la peor ventana en las tres arquitecturas: entre 0.5 y 1.5 MW más de RMSE y hasta 3 veces más tiempo de entrenamiento. Una ventana larga aporta más contexto, pero también más pasos por los que propagar el gradiente, lo que dificulta la optimización.

Aquí la información útil está en las últimas horas y en el calendario. Como la hora del día ya entra como variable, el modelo no necesita 48 horas de historia para saber en qué parte del ciclo diario está. Más ventana no es mejor por defecto: 12-24 h es el punto de equilibrio.

3. ¿Existe sobreajuste?

En los 9 modelos finales, no de forma significativa: el RMSE de validación es solo 1–6 % mayor que el de entrenamiento, y el de prueba es parecido. El riesgo sí existe, y el experimento sin Early Stopping lo demuestra: tras 100 épocas, el RMSE de entrenamiento baja a 21.2 MW mientras el de prueba sube a 36.9 MW. El modelo memoriza el ruido del entrenamiento y deja de generalizar. Early Stopping es lo que lo evita.

4. ¿Qué es y qué efecto tiene Dropout?

Dropout es una técnica de regularización. En cada paso de entrenamiento apaga al azar una fracción de las salidas de una capa (20 % con `Dropout(0.2)`), así la red no puede depender de neuronas concretas y aprende representaciones más generales. Solo actúa durante el entrenamiento; al predecir se usan todas las neuronas.

En la arquitectura C, pasar de Dropout 0.0 a 0.2 y 0.4 bajó el RMSE de prueba de 25.7 a 25.2 y 24.9 MW. La brecha validación/entrenamiento bajó de 1.064 a 1.038. Como Dropout solo actúa al entrenar, la pérdida de entrenamiento queda por encima de la de validación. El efecto es moderado porque Early Stopping ya controlaba el sobreajuste.

5. ¿Qué es y qué efecto tiene Early Stopping?

Early Stopping detiene el entrenamiento automáticamente cuando el error de validación deja de mejorar durante *paciencia* épocas (aquí 10) y restaura los pesos de la mejor época. Tiene tres efectos:

- **Evita el sobreajuste:** sin él, el RMSE de prueba fue 36.9 MW en lugar de 25.7 MW.
- **Ahorra tiempo:** los 9 modelos se detuvieron entre la época 17 y la 32 de un máximo de 100.
- **Elimina la necesidad de adivinar el número de épocas:** cada modelo entrena lo que necesita.

6. ¿Más neuronas implican necesariamente mejores resultados?

No. La LSTM base con 256 unidades (278 785 parámetros) obtuvo un RMSE de prueba de 25.1 MW, peor que con 16 unidades (2 065 parámetros, 24.7 MW), y entrenó unas 5 veces más lento. Entre arquitecturas pasa lo mismo: C (125 249 parámetros) no supera a A (20 545).

Todos los modelos llegan al mismo piso de error, unos 24–25 MW, que también es su error en entrenamiento. Ese piso es la variabilidad aleatoria de la demanda hora a hora, que ninguna variable del dataset explica. Cuando el límite son los datos y no la capacidad del modelo, añadir neuronas solo agrega costo y riesgo de sobreajuste.

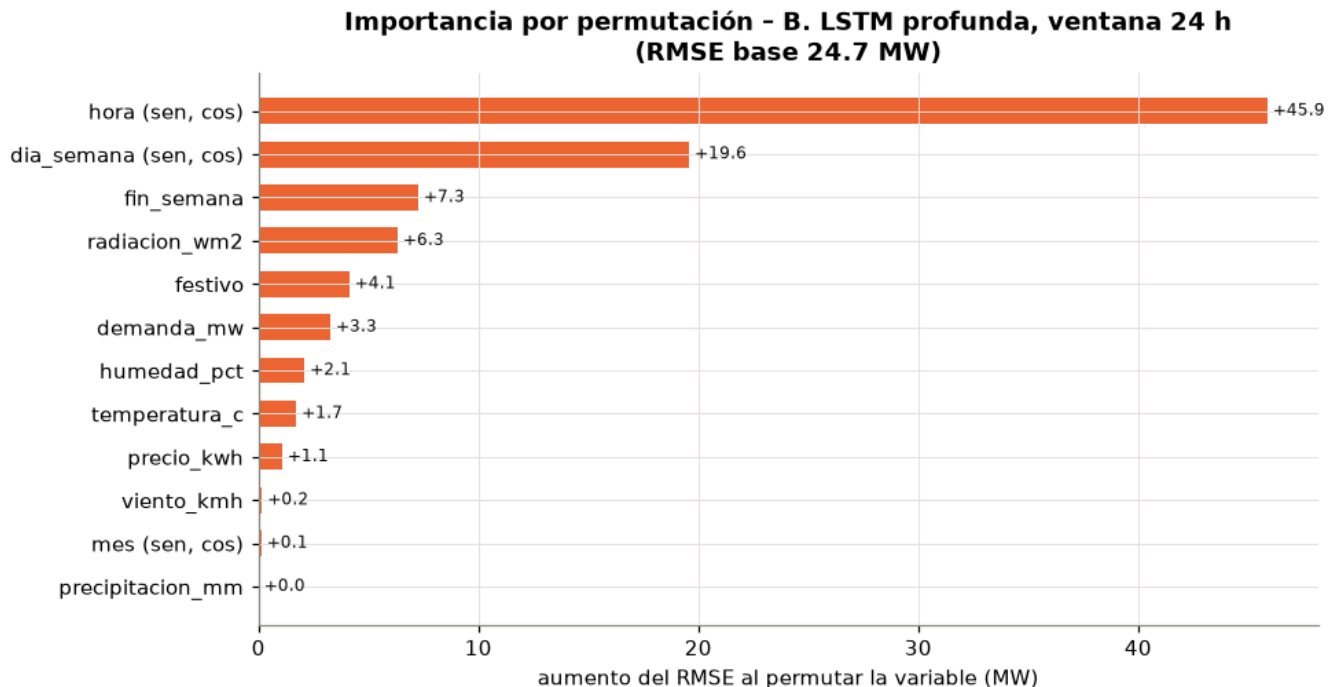
7. ¿Qué modelo recomendaría para producción y por qué?

B. LSTM profunda con ventana de 24 h: MAE 19.6 MW, RMSE 24.7 MW, MAPE 3.36 % y R^2 0.904 en prueba.

- Fue el mejor en validación, que es el criterio correcto de selección, y también el mejor en prueba, así que la elección es consistente.
- Generaliza bien: brecha validación/entrenamiento de 1.04.
- Es ligero: 32 929 parámetros, se entrena en un minuto y predice en milisegundos, lo que facilita reentrenarlo con frecuencia.
- 24 h es una ventana natural para la operación: el día completo anterior.

A con 24 h es una alternativa igual de válida si se prioriza la máxima simplicidad (diferencia de 0.13 MW). No se recomienda C, que tiene 4 veces más parámetros sin ganancia, ni la ventana de 48 h.

8. ¿Qué variables pueden influir más en la demanda?



Se midió cuánto empeora el RMSE del mejor modelo al desordenar cada variable en el conjunto de prueba. La hora del día es, de lejos, la más influyente (+45.9 MW). Le siguen el día de la semana (+19.6 MW), el fin de semana (+7.3 MW), la radiación solar (+6.3 MW) y el festivo (+4.1 MW): el ciclo diario y la actividad laboral dominan la demanda.

La demanda reciente, la humedad y la temperatura aportan menos (+1.7 a +3.3 MW), y el precio, el viento, el mes y la lluvia casi nada. Ojo: si dos variables comparten información, como la temperatura con la hora y la estación, desordenar solo una subestima su importancia.

9. ¿Qué limitaciones tiene el modelo?

- **Solo predice una hora adelante.** Para el día siguiente habría que encadenar predicciones, con acumulación de error, o entrenar un modelo multi-paso.
- **Necesita las mediciones hasta la hora actual.** Si el clima llega tarde o con fallas, el *forward fill* es una solución simple pero no ideal para huecos largos.
- **Festivos:** el error es mayor (MAE 30.7 MW frente a 19.3 MW), porque solo hay 12 días festivos en dos años.
- **Piso de error de unos 25 MW:** bajarlo exige nuevas variables (eventos, actividad industrial, pronósticos del clima), no un modelo más grande.
- **Pocos datos:** dos años, de los que el entrenamiento cubre 1.5 ciclos anuales. Situaciones fuera de ese rango, como olas de calor extremas, son extrapolaciones.
- **Deriva:** si el patrón de consumo cambia, el modelo pierde precisión y debe reentrenarse periódicamente.
- **Interpretabilidad limitada:** una LSTM no explica cada predicción individual.

10. Despliegue: app web

El modelo recomendado está desplegado en una app web hecha con Streamlit (`app.py`): el usuario ingresa las variables independientes (X) y ve la predicción de la demanda de la hora siguiente (y). Se abre con `streamlit run app.py`.

The screenshot shows a web application for predicting electricity demand. On the left sidebar, under 'Modelo', the selected model is 'B. LSTM profunda - ventana 24 h'. Below this, a star icon indicates it's the best model. The 'Desempeño en prueba' section shows metrics for Oct-dic 2026: MAE (MW) 19.6, RMSE (MW) 24.7, MAPE (%) 3.36, and R² 0.904. The main content area has a title 'Predicción de la demanda eléctrica de la próxima hora' and a description: 'El modelo recibe las variables independientes (X) de las últimas 24 horas (demanda, clima, precio y calendario) y predice la demanda de la hora siguiente (y)'. There are three tabs: 'Predicción' (active), 'Comparación de modelos', and 'Cómo funciona'. Section 1, 'Momento de la predicción', includes a date selector '08/12/2026' and a time selector '18:00'. A blue box states: 'Se predice la demanda de martes 08/12/2026 19:00 usando las horas 07/12 19:00 + 08/12 18:00'. Section 2, 'Variables independientes (X)', has two options: 'Formulario (histórico + mis ajustes)' (selected) and 'Cargar un CSV propio'. Below, it says 'Hora actual t = martes 08/12/2026 18:00 - ajusta los valores para simular un escenario:'. There are input fields for: Demanda actual (MW) 579.39, Temperatura (°C) 16.82, Humedad relativa (%) 74.74, Viento (km/h) 9.73, Radiación solar (W/m²) 66.61, Precipitación (mm) 0.00, and Precio de la energía (por kWh) 32.32. A 'Día festivo' checkbox is checked. At the bottom, it says 'Calendario derivado automáticamente de la fecha: martes, hora 18, mes 12, fin de semana: no.' and a link to 'Ver o editar las 23 horas anteriores (t-23 ... t-1)'. A 'Deploy' button is in the top right.

1. **Elegir el modelo** en la barra lateral: cualquiera de los 9, con el recomendado (B, 24 h) por defecto, y sus métricas en prueba.
2. **Elegir la fecha y la hora actual (t)**. La app carga automáticamente las 24 horas anteriores.
3. **Ingresar o ajustar las X** de la hora actual: demanda, temperatura, humedad, viento, radiación, precipitación, precio y festivo. El calendario (hora, día, mes, fin de semana) se deriva de la fecha. Opcionalmente se pueden editar las horas anteriores, sumar grados a toda la ventana para simular un escenario, o cargar un CSV propio.
4. **Ver la predicción (y)** para $t+1$. Si esa hora existe en el dataset, la app muestra también la demanda real y el error.



- **Ejemplo verificado:** para el 08/12/2026 a las 19:00 (festivo), la app predijo 639.2 MW y la demanda real fue 642.0 MW (error de 2.7 MW, 0.4 %).
- **Escenario:** al subir la temperatura de la hora actual a 30 °C, la predicción pasa a 647.4 MW (captura de arriba).
- **Mismo preprocesamiento:** la app y el notebook comparten el módulo `lstm_pipeline.py` y los parámetros de normalización de entrenamiento, por lo que la app da exactamente la misma predicción que el notebook.
- **Validaciones:** la app rechaza entradas imposibles (humedad fuera de 0-100, valores negativos) y exige horas consecutivas en el CSV.

11. Conclusiones

- **Las tres arquitecturas LSTM predicen la demanda de la próxima hora con un error de un 3.4 % (R² de 0.90) y reducen en un 44 % el error de la persistencia.** Lo permiten unos datos limpios y una preparación correcta: split cronológico, normalización solo con entrenamiento y ninguna variable del futuro como entrada.

- **La arquitectura importa poco en este problema:** los 9 modelos quedan dentro de 1.5 MW de RMSE entre sí. Importan más la ventana (48 h empeora) y, sobre todo, la regularización: sin Early Stopping el error de prueba sube un 44 %.
- **Más complejidad no es mejor:** más neuronas o capas no bajan el error por debajo del piso de unos 25 MW que imponen los datos. Para mejorar harían falta nuevas variables.
- **Modelo recomendado:** B. LSTM profunda con ventana de 24 h (MAE 19.6 MW, RMSE 24.7 MW, MAPE 3.36 %, R^2 0.904), desplegado en la app web.